

[CaiTien.md](#) - Roadmap nâng cấp MarketLens thành CEM-RAG hướng ESWA

Tài liệu này mô tả cách nâng cấp project MarketLens

1. Cơ sở đọc tài liệu và hiện trạng repo

Tài liệu này được tổng hợp từ:

[MoTa.md](#): định hướng CEM-RAG, event memory, learned retrieval, calibrated trading policy.

[README.md](#): kiến trúc microservice tổng

quan.

[architecture.md](#): hợp đồng module, data models, inter-module communication, testing strategy.

docs/[api-guide.md](#): API đang triển khai trên VPS, workflow run/backfill/result.

docs/[backtest_report_2026-05.md](#): backtest BTC hiện tại giai đoạn [2022-01-01](#) đến [2026-05-17](#).

Code hiện tại trong stockmem, aihub, factor_ledge, main_controller, market_data, crawler, shared.

Các nguồn học thuật và guideline dùng để định vị hướng nâng cấp:

ESWA Aims & Scope: <https://www.sciencedirect.com/journal/expert-systems-with-applications>

ESWA Guide for Authors: <https://www.sciencedirect.com/journal/expert-systems-with-applications>

[systems-with-applications/publish/guide-for-authors](#)

FinBERT: <https://arxiv.org/abs/1908.10063>

PatchTST: <https://arxiv.org/abs/2211.14730>

StockMem event-reflection memory: <https://arxiv.org/abs/2512.02720>

FinSeer financial time-series RAG: <https://arxiv.org/abs/2502.05878>

Dissemination-aware FinGPT: <https://openreview.net/forum?id=I2nHuTk6nc>

CryptoLin corpus: <https://link.springer.com/article/10.1007/s10579-024-09743-x>

ESWA phù hợp với hướng này vì journal tập trung vào expert/intelligent systems, finance, information retrieval, stock trading, neural networks, knowledge discovery, data mining và text mining. Vì vậy MarketLens không nên được trình bày như một chatbot hoặc pipeline LLM đơn thuần, mà nên được nâng cấp và viết thành một intelligent financial forecasting system có

dữ liệu point-in-time, event memory, retrieval được học từ outcome, mô hình dự báo xác suất, kiểm định thống kê và phần mềm có thể tái lập.

2. Hiện trạng MarketLens

MarketLens hiện là pipeline crypto news intelligence dạng microservice:

Service	Vai trò hiện tại
Crawler	Lấy tin tức từ Supabase/RSS, trả article có sentiment/factors/raw text.
AIHub	Sentiment, factor extraction RAG

	extraction, RAG prediction qua LLM.
MarketData	OHLCV, history, indicators từ Binance/ TradingView.
StockMem	Lưu daily record và tìm similar cases bằng weighted similarity.
FactorLedge	Chuẩn hóa factor, classify factor thành vector taxonomy 75d.
MainController	Điều phối collect -> score -> StockMem -> predict -> result.
LLMGateway	Final BUY/HOLD/ SELL decision và guardrail policy.

Luồng hiện tại:

text



Crawler + MarketData

|

v

MainController step_ai_score

|

v

FactorLedge factor vector + StockMem
daily record

|

v

StockMem weighted kNN search

|

v

AIHub/LLMGateway RAG predict

|

v

PredictionResult { signal, confidence,
explanation, similar_cases }

StockMem hiện dùng split-vector embedding:

factor_vec: 75 chiều, gồm 62 event type bits + 13 group bits.

indicator_vec: 5 chiều, gồm MSI, RSI, sentiment, Fear & Greed, price_change.

price_vec: 60 chiều, gồm return/range/volume change của 20 candles.

Similarity hiện tại:

text



```
score = w_factor * cosine(factor_vec)
      + w_indicator * cosine(indicator_vec)
      + w_price * cosine(price_vec)
```

Repo đã có các điểm tốt:

[StockMem.search](#) đã có before_date để tránh retrieve record cùng ngày hoặc tương lai trong backtest.

stockmem/scripts/[optimize_weights.py](#) đã có Bayesian/Optuna weight optimization.

main_controller/src/orchestrator/[steps.py](#) đã có historical mode as_of_date.

scripts/[backtest_runner.py](#) đã có backtest theo ngày và forward returns 1d/3d/7d/15d/30d.

docs/[backtest_report_2026-05.md](#) đã có báo cáo thực nghiệm BTC tương đối rõ.

Tuy nhiên, repo hiện chưa có implementation CEM-RAG đầy đủ. Cụ thể:

Chưa có aihub/src/predict/[cem_rag.py](#).

Chưa có schema EventRecord,

DailyEventState, RetrievalTrace.

Chưa có event memory point-in-time theo ngày/symbol.

Chưa có learned retriever được train bằng supervised contrastive/triplet learning.

Chưa có p_up, p_down, p_hold được calibration.

Chưa có evaluation suite đầy đủ cho MCC, Macro-F1, Brier, ECE, Sharpe, Sortino, max drawdown, McNemar, block bootstrap.

3. Kết quả backtest hiện tại và ý nghĩa

Theo docs/[backtest_report_2026-05.md](#):

Scope: 1,568 backtests, BTC, [2022-01-01](#) -> [2026-05-17](#).

Pipeline: MainController -> LLMGateway ->

StockMem kNN -> Regime Guardrails.

Signal distribution:

BUY: 399 ngày, 25.4%.

SELL: 170 ngày, 10.8%.

HOLD: 999 ngày, 63.7%.

Coverage BUY+SELL: 36.3%.

Kết quả D+7:

Signal	Accuracy D+7	Nhận xét
BUY	51.9%	Chỉ nhỉnh hơn random, tốt hơn ở bull market 2023-2024.
SELL	39.4%	Yếu nhất; false SELL nhiều trong

		bull run.
HOLD	7.1%	HOLD chỉ đúng khi $\text{abs}(\text{return})$ nhỏ, nên metric thấp.
Directional precision BUY+SELL	48.2%	Chưa đủ mạnh cho paper.

Điểm yếu chính:

Final decision phụ thuộc nhiều vào LLM + guardrails hardcoded, khó chứng minh tối ưu.
Confidence chưa calibration tốt; SELL đúng và SELL sai có confidence gần nhau.

HOLD quá nhiều, coverage thấp.

SELL overall dưới 50%, không đủ thuyết phục.

Backtest hiện tại chưa đủ baseline học thuật

như XGBoost, PatchTST, FinBERT text-only, fixed kNN, learned retriever.

Điểm sáng quan trọng từ MoTa.md:

Kết quả kNN>Returns nội bộ cho thấy historical memory có tín hiệu dự báo tốt hơn LLM decision hiện tại.

Cấu hình New Bayesian threshold $\pm 2\%$ trong MoTa.md đạt BUY DA khoảng 59.7%, SELL DA khoảng 57.5%, coverage khoảng 58.2%.

Các số liệu này nên được xem là milestone nội bộ cần tái lập bằng artifact chính thức, chưa nên tuyên bố là final paper result nếu chưa có split train/validation/test và leakage audit.

Kết luận hiện trạng:

MarketLens có kiến trúc tốt để phát triển thành một expert/intelligent system, nhưng research core hiện tại chưa đủ mạnh để nộp ESWA. Hướng nâng cấp nên chuyển từ "LLM + fixed kNN + guardrails" sang "point-in-time event memory + learned retrieval + calibrated probabilistic forecasting + rigorous evaluation".

4. Định vị paper đề xuất

Tên ngắn:

text



CEM-RAG: Crypto Event Memory
Retrieval-Augmented Forecasting

Tên đầy đủ:

text



CEM-RAG: A Point-in-Time Crypto Event
Memory and Learned Retrieval
Framework
for Multi-Horizon Cryptocurrency
Movement Forecasting

Nhiệm vụ nghiên cứu:

Primary task: dự báo hướng biến động
crypto 7 ngày.

Secondary horizons: 1d, 3d, 15d, 30d.

Output model: xác suất UP, DOWN, HOLD/
NO_TRADE.

Output trading: BUY/SELL/HOLD bằng policy
tune trên validation.

Output explainability: top events, retrieved

historical cases, evidence trace.

Research questions:

RQ	Câu hỏi
RQ1	Event memory có cải thiện crypto movement forecasting so với price-only và sentiment-only không?
RQ2	Learned retriever có lấy historical cases tốt hơn fixed weighted kNN không?
RQ3	Fusion price-event-retrieval có tốt hơn

	từng modality riêng lẻ không?
RQ4	Calibration + validation-tuned trading policy có cải thiện Sharpe sau phí không?

Hypotheses:

- H1: CEM-RAG có MCC cao hơn baseline mạnh nhất ít nhất 0.05 trên held-out test.
- H2: CEM-RAG có net Sharpe cao hơn baseline mạnh nhất ít nhất 0.25 sau transaction cost.
- H3: Bỏ event memory làm giảm MCC/Macro-F1 rõ rệt.
- H4: Learned retriever vượt random, BM25/ text embedding, numeric Euclidean và fixed

weighted kNN.

5. Roadmap nâng cấp chi tiết

Bước 1: Chuẩn hóa protocol dữ liệu point-in-time

Mục tiêu:

Mọi prediction row phải biết chính xác hệ thống được phép nhìn thấy gì tại thời điểm dự báo.

Loại bỏ data leakage trước khi train model.

Thay đổi cần làm:

1. Thêm cutoff_time vào mọi artifact prediction/backtest.
2. Chuẩn hóa article fields:
 - published_at
 - crawled_at
 - source
 - url
 - canonical_url
 - content_hash
 - symbol_mentions
3. Với ngày T, chỉ dùng article thỏa:

text



```
published_at <= cutoff_time  
crawled_at <= cutoff_time
```

4. Với market candle, chỉ dùng candle đã đóng trước hoặc tại cutoff hợp lệ.

2. Với retrieval, bắt buộc:

text



```
candidate.date < query.date
```

```
candidate.date + horizon <= query.date
```

Điểm cần sửa trong code sau này:

shared.models.memory.StockMemRecord:
thêm optional cutoff_time.

scripts/[backtest_runner.py](#): ghi cutoff_time
vào output/backtest_results.

[StockMem.search](#): giữ before_date; thêm
kiểm tra maturity theo horizon trong
evaluation layer.

Dataset builder mới không được fit scaler
trên toàn bộ dữ liệu; scaler chỉ fit trên train.

Leakage checklist bắt buộc:

text



- [] Mỗi prediction row có cutoff_time.
- [] Mỗi article có published_at và crawled_at.
- [] Article sau cutoff không xuất hiện trong features.
- [] Feature normalization fit trên train only.
- [] Validation chỉ dùng tune hyperparameter, tau, calibration.
- [] Test locked và chỉ evaluate một lần cho result chính.
- [] Retrieval không lấy record cùng ngày hoặc tương lai.
- [] Retrieval không dùng future_return chưa matured tại prediction date.
- [] Duplicate URL/content được dedup trước split.
- [] Transaction cost được trừ trong trading metrics.

Temporal split đề xuất:

text



Train: 2018-01-01 -> 2023-12-31

Validation: 2024-01-01 -> 2024-12-31

Test: 2025-01-01 -> 2026-05-17

Nếu dữ liệu trước 2022 chưa đủ:

text



Train: 2022-01-01 -> 2023-12-31

Validation: 2024-01-01 -> 2024-12-31

Test: 2025-01-01 -> 2026-05-17

Không shuffle dữ liệu time-series.

Bước 2: Xây event extraction từ article/factor

Mục tiêu:

- Biến news/factor rời rạc thành event có cấu trúc, có thể kiểm định.
- Giảm phụ thuộc vào LLM final reasoning.

Schema đề xuất:

python



```
class EventRecord(BaseModel):
    event_group: str
    event_type: str
    entities: list[str]
    polarity: float
    confidence: float
    source_article_id: str
    observed_at: datetime
    evidence_text: str | None = None
```

Nguồn input:

- IngestionRecord.article_name
- IngestionRecord.summary
- IngestionRecord.raw_text
- IngestionRecord.factors
- FactorLedge taxonomy 13 group/62 type hiện có.

Event taxonomy cần mở rộng cho crypto:

Group	Event type ví dụ
Regulation & Legal	ETF approval, ETF delay, enforcement action, sanctions, staking regulation

Market Performance	ETF flow, volume surge, BTC dominance shift, liquidation cascade
Whale & On-chain	exchange inflow, exchange outflow, whale accumulation, miner selling
Risk & Warning	bridge exploit, stablecoin depeg, exchange insolvency, proof-of-reserves concern
Protocol & Product	protocol upgrade, mainnet launch, fee/gas change, supply dynamics
Macroeconomic	CPI surprise, Fed rate decision, DXY movement, Treasury

Triển khai đề xuất:

Tạo module mới trong aihub/src/events/:

[schema.py](#): EventRecord,
EventExtractionRequest,
EventExtractionResponse.

[extractor.py](#): map article/factor -> event.

[taxonomy.py](#): taxonomy crypto event.

[prompts.py](#): prompt event extraction nếu
dùng LLM.

API mới trong aihub/src/[api.py](#):

text



POST /events/extract

Payload:

json



```
{  
  "symbol": "BTC",  
  "article": {  
    "id": "66214",  
    "title": "SEC delays Ethereum ETF  
decision",  
    "published_at":  
"2026-05-04T06:17:30Z",  
    "source": "cointelegraph.com",  
    "summary": "...",  
    "raw_text": "..."  
  }  
}
```

Response:

json



```
{
  "events": [
    {
      "event_group": "regulation_legal",
      "event_type": "etf_delay",
      "entities": ["SEC", "Ethereum ETF",
"ETH"],
      "polarity": -0.72,
      "confidence": 0.84,
      "source_article_id": "66214",
      "observed_at":
"2026-05-04T06:17:30Z",
      "evidence_text": "SEC delays
Ethereum ETF decision"
    }
  ]
}
```

Testing:

Unit test deterministic mapping cho known headlines.

Test polarity theo asset:

SEC sues Binance bearish cho BNB, có thể neutral hoặc mixed cho BTC.

Stablecoin exchange inflow rising có thể bullish nếu liquidity inflow, bearish nếu panic context.

Test confidence nằm trong $[0, 1]$, polarity trong $[-1, 1]$.

Bước 3: Cluster same-day news thành DailyEventState

Mục tiêu:

Một ngày có 30 bài cùng nói ETF inflow không nên là 30 tín hiệu độc lập.

Cần đo breadth, source diversity và novelty để phân biệt event mới với tin lặp lại.

Schema đề xuất:

python



```
class DailyEventState(BaseModel):  
    date: date  
    symbol: str  
    events: list[EventRecord]  
    article_count: int  
    source_count: int  
    source_diversity: float  
    novelty_7d: float  
    novelty_30d: float  
    dominant_event_groups: list[str] = []
```

Aggregation logic:

1. Lấy tất cả article của symbol trong ngày T trước cutoff.
2. Extract event cho từng article.

3. Group event theo:

text



(event_group, event_type,
normalized_entities)

4. Với mỗi cluster, tính:

article_count

source_count

source_diversity

weighted polarity trung bình

confidence trung bình hoặc max-

confidence weighted average

2. Tính novelty:

text



$\text{novelty}_{7d} = 1 -$

```
novelty_7d = 1 -  
max_similarity(current_event_cluster,  
clusters in T-7..T-1)  
novelty_30d = 1 -  
max_similarity(current_event_cluster,  
clusters in T-30..T-1)
```

6. Persist DailyEventState để dùng train/evaluate.

Feature vector event đề xuất:

text



```
event_type multi-hot  
event_group multi-hot  
mean_polarity  
max_abs_polarity  
article_count_log  
source_count_log  
source_diversity  
novelty_7d  
novelty_30d  
confidence_mean  
top_event_breadth
```

Ví dụ:

json



```
{
  "date": "2025-03-18",
  "symbol": "BTC",
  "article_count": 28,
  "source_count": 11,
  "source_diversity": 0.86,
  "novelty_7d": 0.74,
  "novelty_30d": 0.54,
  "events": [
    {
      "event_group":
"market_performance",
      "event_type": "etf_flow",
      "entities": ["BlackRock", "Bitcoin ETF",
"BTC"],
      "polarity": 0.82,
      "confidence": 0.88,
      "source_article_id":
```

```
"cluster:2025-03-18:etf_flow",  
  "observed_at":  
    "2025-03-18T23:59:59Z"  
}  
]  
}
```

Testing:

20 bài cùng event -> 1 cluster chính, không
20 event độc lập.

20 bài từ 1 source có source diversity thấp
hơn 20 bài từ nhiều source.

Event lặp lại 30 ngày có novelty thấp.

Event shock mới có novelty cao.

Bước 4: Mở rộng StockMem từ factor memory

sang event memory

Mục tiêu:

StockMem hiện đang lưu factor/indicator/price. Cần thêm event-memory features để phục vụ CEM-RAG.

Thay đổi schema:

python



```
class StockMemRecord(BaseModel):  
    ...  
    event_state: DailyEventState | None =  
    None  
    event_vector: list[float] = []  
    cutoff_time: datetime | None = None
```

Thay đổi embedder:

May doi embedder.

Hiện tại:

text



factor_vec 75d
indicator_vec 5d
price_vec 60d

Đề xuất:

text



event_vec d_event
factor_vec 75d
indicator_vec 5d
price_vec 60d

Search score v1:

text



```
score = w_event * sim(event_vec)
        + w_factor * sim(factor_vec)
        + w_indicator * sim(indicator_vec)
        + w_price * sim(price_vec)
```

Yêu cầu giữ compatibility:

Nếu event_vector rỗng thì fallback về scoring cũ.

Không phá endpoint /record và /search hiện tại.

Cần version hóa retriever để backtest biết đang dùng method nào.

API search đề xuất:

python



```
class SearchRequest(BaseModel):  
    query: StockMemRecord  
    k: int = 5  
    before_date: date | None = None  
    horizon: str = "7d"  
    retriever_type: str = "fixed_knn" #  
    fixed_knn | learned_cem
```

Response mở rộng:

python



```
class RetrievalTrace(BaseModel):  
    retrieved_date: date  
    similarity: float  
    event_match: dict[str, float]  
    future_return_7d: float | None  
    retriever_version: str
```

Testing:

Search cùng symbol vẫn được giữ.

before_date loại record cùng ngày/tương lai.

retriever_type=fixed_knn trả kết quả như cũ khi chưa bật learned retriever.

Event vector rỗng không làm crash search.

Search response có retriever version để audit.

Bước 5: Huấn luyện learned event-memory retriever

Mục tiêu:

Thay weighted cosine thủ công bằng retriever học từ outcome.

Retriever phải phân biệt được trường hợp

keyword giống nhau nhưng outcome trái chiều.

Training data:

Mỗi row:

json



```
{  
  "date": "2024-03-04",  
  "symbol": "BTC",  
  "event_vector": [0.1, 0.0, 1.0],  
  "factor_vec": [0, 1, 0],  
  "indicator_vec": [0.55, 0.22],  
  "price_vec": [0.01, -0.02],  
  "future_return_7d": 6.4,  
  "regime": "bull",  
  "cutoff_time": "2024-03-04T23:59:59Z"  
}
```

Positive pair:

Cùng hướng future return 7d.

Regime volatility gần nhau.

Event group/type tương đồng hoặc financial narrative tương đồng.

Hard negative:

Event text/taxonomy rất giống query.

Nhưng future return 7d trái dấu.

Ví dụ:

text



Query:

ETF optimism + RSI 58 + volume up ->
future_return_7d +6%

Positive:

ETF inflow + volume breakout ->
future_return_7d +8%

Hard negative:

ETF approval + RSI 85 + sell-the-news ->
future_return_7d -7%

Loss đề xuất:

text



Triplet loss:

$\max(0, \text{margin} + \text{dist}(\text{query}, \text{positive}) - \text{dist}(\text{query}, \text{negative}))$

hoặc supervised contrastive loss:

pull same-direction/same-regime cases together,
push opposite-outcome hard negatives apart.

Files/scripts đề xuất:

stockmem/src/search/[learned_retriever.py](#)
stockmem/scripts/[build_cem_dataset.py](#)
stockmem/scripts/[train_learned_retriever.py](#)
stockmem/scripts/[evaluate_retriever.py](#)
output: stockmem/config/
learned_retriever.json hoặc model
checkpoint trong artifacts/models/.

Metrics retriever:

Metric	Ý nghĩa
Hit same direction@5	Top-5 có cùng direction với query bao nhiêu lần.
Avg retrieved sign	Mean sign của

accuracy	retrieved future returns đúng hướng không.
Hard-negative separation	Query gần positive hơn hard negative bao nhiêu.
Retrieval trace quality	Case lấy ra có narrative tài chính hợp lý không.

Acceptance gate retriever:

Learned retriever phải vượt fixed weighted
kNN trên validation và test.

Nếu chỉ tốt trên train, không dùng làm
contribution chính.

Nếu learned retriever không vượt kNN-
Returns, paper nên trung thực gọi kNN-

Returns là baseline mạnh nhất.

Bước 6: Xây forecasting model price-event-retrieval fusion

Mục tiêu:

Model dự báo xác suất thay vì để LLM quyết định BUY/SELL/HOLD.

LLM chỉ nên hỗ trợ extraction/explanation, không là final decision không kiểm chứng.

Input:

1. Price encoder:

30 ngày OHLCV.

Return, range, volume change.

RSI, MACD, Bollinger, Fear & Greed, MSI.

2. Event encoder:

Event taxonomy vector.

Event text embedding từ FinBERT/

CryptoBERT/LedgerBERT nếu có.

polarity, confidence, article_count,

source_count, novelty_7d, novelty_30d.

3. Retrieval encoder:

Top-k historical cases.

Similarity.

Event match score.

Historical future returns chỉ dùng cho
retrieved past cases đã matured.

Fusion:

text



price tokens -> temporal encoder, ví dụ PatchTST-style
event tokens -> event encoder
retrieval tokens -> case encoder

|
v
cross-attention / gated fusion

|
v
p_up, p_down, p_hold

Output schema đề xuất:

python



```
class
CEMRAGPredictResponse(BaseModel):
    horizon: str
```

```
horizon: str
p_up: float
p_down: float
p_hold: float
signal: SignalType
policy_tau: float
confidence: float
explanation: str
top_events: list[EventRecord]
retrieval_trace: list[RetrievalTrace]
```

Policy:

text



BUY nếu $p_{up} - p_{down} \geq \tau$
SELL nếu $p_{down} - p_{up} \geq \tau$
HOLD ngược lại

Quy tắc:

tau chỉ tune trên validation.
Không tune tau trên test.
Horizon primary: 7d.
1d/3d/15d/30d là auxiliary hoặc secondary analysis.

Triển khai theo mức độ:

Mức	Mô tả	Khi nào dùng
v0	kNN>Returns deterministic + calibrated policy	Baseline mạnh, dễ tái lập.
v1	Gradient boosting/ logistic trên event+price+retrieval features	Nhanh, ít rủi ro.

v2

Neural fusion
với te

